

WebSockets & Socket.IO

Interview Q&A

Everything behind the chat app — the protocol, the handshake, rooms, delivery guarantees, acks, and scaling. Written as answers you can say out loud.

Protocol internals

Socket.IO

Rooms & auth

Delivery & acks

Scaling

CONTENTS

1. Protocols & the network
2. HTTP vs WebSocket
3. The WebSocket handshake
4. Real-time options compared
5. Raw ws vs Socket.IO
6. CORS & why it exists
7. Connections vs people · rooms
8. Auth & trusting the client
9. Emit variants
10. Namespaces vs rooms
11. Heartbeat & disconnects
12. Delivery guarantees
13. Acknowledgements (acks)
14. Scaling to many servers
15. When NOT to use WebSockets
16. Rapid fire

1. Protocols & the network

The foundation. Real-time only makes sense once you know what a connection actually is.

1.1 What is a protocol?

An agreed set of rules for how two machines talk — the format of messages and the order they come in. HTTP, WebSocket, TCP, IP are all protocols. Like a shared language: both sides must speak it or the bytes are meaningless.

1.2 IP vs port vs connection?

- **IP** identifies the *machine*.
- **Port** identifies a *program* on that machine (a server listens on one, e.g. 5000).
- **Connection** is a live conversation between two of them.

A connection is identified by a **four-tuple**: (source IP, source port, destination IP, destination port). Change any one and it's a different connection.

FOLLOW-UP · "HOW DO 3 TABS TO THE SAME SERVER STAY SEPARATE?"

Same destination IP+port, but each tab gets a **different source port**, so all four-tuples are unique. That's how the server keeps them apart.

1.3 What are ephemeral ports?

When your browser makes a connection, the OS picks a temporary **source** port from a high range (roughly 49152–65535). The server's port is fixed and known; the client's is throwaway, assigned per connection.

1.4 TCP vs UDP?

TCP — reliable, ordered, connection-based. Delivers every byte in order or retries. Used by HTTP and WebSocket. **UDP** — fire-and-forget, no ordering or delivery guarantee, but faster. Used for video calls, games, DNS, and HTTP/3 (over QUIC).

2. HTTP vs WebSocket

Why the web needed a second protocol for real-time.

2.1 Why can't plain HTTP do real-time?

HTTP is **request/response** and client-initiated. The server can't speak unless the client asks first, and each exchange is a fresh round trip with full headers. There's no way for the server to *push* — which is exactly what chat, notifications, and live data need.

2.2 Define WebSocket for an interviewer.

"A protocol that upgrades a single HTTP connection into a **persistent, two-way (full-duplex)** channel over the same TCP connection. After the handshake, either side can send a message any time, with almost no overhead per message — no new request, no repeated headers."

3. The WebSocket handshake

How a normal HTTP request becomes a WebSocket. Interviewers love this one.

3.1 Walk through the handshake.

1. Client sends a normal HTTP `GET` with special headers: `Upgrade: websocket` , `Connection: Upgrade` , and a random `Sec-WebSocket-Key` .
2. If the server agrees, it replies `101 Switching Protocols` .
3. The **same TCP connection** is now spoken in the WebSocket protocol instead of HTTP — no second connection is opened.

KEY PHRASE

"It starts as HTTP and is upgraded in place. One TCP connection, new language after 101."

3.2 What does "once upgraded, it's only a WebSocket" mean?

After the `101` , that connection can no longer carry HTTP requests. The bytes flowing over it are WebSocket **frames**, not HTTP messages. It stopped being an HTTP conversation and became a WebSocket one — same pipe, different rules.

3.3 `ws://` vs `wss://` ? What are frames?

`ws://` is plaintext; `wss://` is WebSocket over TLS (encrypted), the equivalent of `https` . Always use `wss` in production.

Data travels in **frames** — small packets with an *opcode* saying what they are: text, binary, ping, pong, or close. Ping/pong frames are how the connection is health-checked.

4. Real-time options compared

The classic "WebSocket vs SSE vs polling" question, from crude to full-duplex.

4.1 Short polling vs long polling?

Short polling — client asks "anything new?" on a timer. Simple, but most responses are empty (wasteful) and you're seconds behind.

Long polling — server holds the request open until it has data, sends **one** message, then the connection **closes** and the client immediately reopens. Near real-time, but one message per request and constant churn.

4.2 Isn't long polling the same as SSE — both hold the request open?

No — the difference is **messages per request**. Long polling holds it open for *one* answer, then closes and reopens. **SSE streams many** messages down the *same* open connection, never closing between them.

PHONE ANALOGY

Long polling = they tell you one thing then hang up, you call back. SSE = you call once and they keep talking without hanging up.

4.3 What is SSE and when do you use it?

Server-Sent Events — one long-lived HTTP response the server keeps writing to. **One-way** (server → client), auto-reconnects natively (with `Last-Event-ID` to resume), runs over plain HTTP with no Upgrade. Text only. Perfect for notifications, live feeds, tickers, progress bars.

4.4 The decision, in one line?

SAY THIS

"If only the server pushes, SSE is simpler and cheaper. If both sides talk, WebSocket. Polling is the fallback when you can't hold a connection open."

5. Raw ws vs Socket.IO

They are not the same thing, and not interoperable.

5.1 Is Socket.IO just WebSockets?

No. Socket.IO is a **library with its own protocol layered on top of** WebSocket (and long-polling as fallback). A raw `ws` client **cannot** talk to a Socket.IO server and vice-versa — the message format differs. Socket.IO adds: auto-reconnect, rooms, named events, acknowledgements, heartbeat, multiplexed namespaces, and a polling fallback.

5.2 Why pick raw ws vs Socket.IO?

Raw ws — lightweight, you control everything, but you build reconnect/rooms/acks yourself. **Socket.IO** — batteries included; you get the hard parts free at the cost of a heavier client and its own protocol.

6. CORS & why it exists

6.1 If the server sees the source IP, why does CORS exist?

Because IP identifies the *machine*, not *which web page's JavaScript* is making the request. A malicious page on `evil.com`, open in your browser, sends a request with your **same IP and your valid cookies**. The server can't tell it apart by IP.

CORS uses the `origin` header — set by the browser, not fakeable by page JS — to say "which site's code is this from," so the server can allow or reject it. It defends against a different threat than IP identification.

6.2 Does Socket.IO need its own CORS config?

Yes. The Socket.IO handshake bypasses your Express middleware, so you configure CORS on the `Server` itself:

```
const io = new Server(server, {
  cors: { origin: "http://localhost:3000", credentials: true }
});
```


7. Connections vs people · rooms

The single most important mental model for a real chat app.

7.1 `socket.id` vs a database user id?

`socket.id` identifies **one connection** — it changes on every reconnect and a person with two tabs has two of them. A database `_id` identifies the **person** — permanent. Never key long-lived data (conversations, presence) on `socket.id`.

ONE-LINER

"`socket.id` is a connection, not a person — so you room by user id."

7.2 What is a room?

A **server-side label** attached to sockets. The browser never knows rooms exist. Under the hood it's a map of `roomName` → Set of `socket ids`.

```
socket.join("room-42");
io.to("room-42").emit("event", data); // to everyone wearing the label
socket.leave("room-42");
```

Every socket **auto-joins a room named after its own `socket.id`** — which is why

`io.to(someSocketId).emit(...)` targets exactly one connection.

7.3 Why room by user id instead of socket id?

Because you want to reach a *person*, not a connection. Join a room named after the user's `_id`:

```
socket.join(user._id); // on connect
io.to(user._id).emit("message:new", msg); // reaches ALL their tabs
```

Now five tabs are five sockets in one room. This is what makes WhatsApp Web and your phone stay in sync — and it survives reconnects, because the socket re-joins the room.

8. Auth & trusting the client

8.1 How does the server learn who a connecting socket is?

Best: in the **handshake auth**, sent at connection time — so the server knows immediately, no extra event.

```
// client
socket.auth = { _id: user._id, name };
socket.connect();

// server
io.on("connection", (socket) => {
  const { _id, name } = socket.handshake.auth;
  if (!_id) return socket.disconnect();
  socket.data.userId = _id;      // stash it on the socket
  socket.join(_id);
});
```

`socket.auth` is auto-resent on every reconnect, so identity survives drops for free — unlike a manual `user:join` event you'd have to re-emit.

8.2 Where should `from` come from on a message?

From the **server's record of the socket** (`socket.data.userId`), never from the client's payload. The client sends only `{ to, text }`. If the client supplies `from`, anyone can forge a message as someone else.

SAME RULE AS EXPRESS

" `req.body` for what they're acting on, `req.user._id` for who they are. Never let the client name itself."

REAL APPS

The id should come from a *verified* session cookie or JWT decoded in a Socket.IO middleware (`io.use(...)`), not a plain value the client hands over. A lab sends it directly; production verifies it.

8.3 Async socket handlers — the trap.

Socket.IO does **not** catch errors from your handler. An `async` handler that rejects becomes an **unhandled rejection** — and if your server exits on that, one bad DB write kills everyone's connection. Every async socket handler needs its own `try/catch`. There is no `next` and no error middleware like Express has.

9. Emit variants

Who receives it — especially whether the sender is included.

9.1 The five you must know.

```
socket.emit("e", d)           // only the sender
io.emit("e", d)               // everyone, sender included
socket.broadcast.emit("e", d) // everyone EXCEPT the sender
io.to(room).emit("e", d)      // a room (sender included if in it)
socket.to(room).emit("e", d)  // a room, EXCEPT the sender
```

PATTERN

" socket. narrows to *not me*; io. means *everyone in scope*."

10. Namespaces vs rooms

10.1 What's the difference?

Namespace — a separate channel with its own handlers and middleware, multiplexed over the same connection. For separating *features or access levels* (`/admin` , `/chat`). You define them in code.

Room — a dynamic subgroup *inside* a namespace, purely a broadcast target, created and destroyed at runtime. For picking an *audience*.

SAY THIS

"Namespace = a different endpoint with its own logic. Room = a dynamic subgroup you broadcast to. One socket lives in one namespace but many rooms."

11. Heartbeat & disconnects

11.1 How are dead connections detected?

A TCP connection can die **silently** — laptop sleeps, wifi drops — and still look alive. So Socket.IO runs a **heartbeat**: every `pingInterval` (~25s) the server pings; the client must pong within `pingTimeout` (~20s), or the socket is declared dead and `disconnect` fires with reason *ping timeout*.

11.2 Do raw WebSockets have heartbeats too?

The **protocol provides the tool** (ping/pong control frames) but **not the strategy**. With raw `ws` you write the loop yourself:

```
wss.on("connection", (ws) => {
  ws.isAlive = true;
  ws.on("pong", () => { ws.isAlive = true; });
});
setInterval(() => {
  wss.clients.forEach((ws) => {
    if (!ws.isAlive) return ws.terminate(); // dead → drop
    ws.isAlive = false;
    ws.ping();
  });
}, 30000);
```

Socket.IO's heartbeat is **on by default** — you only tune it. Raw `ws`, you build it.

12. Delivery guarantees

A senior-level tell. Be precise.

12.1 Does Socket.IO guarantee delivery?

No. It guarantees **ordering per connection** — messages from one socket arrive in the order sent — but **not delivery**. Default is **at-most-once**: if the client is disconnected when the server emits, that message is *silently lost*.

ORDERING IS "PER CONNECTION"

Only within one socket. Two tabs = two sockets = no order guarantee between them. After a reconnect it's a new socket, so order across the drop isn't guaranteed either.

12.2 Name the three delivery semantics.

- **At-most-once** — zero or one time, never duplicated, can be lost. (Socket.IO default.)
- **At-least-once** — never lost, can arrive twice (retries).
- **Exactly-once** — the ideal; usually *faked* with at-least-once + dedupe.

12.3 How do you make it reliable?

1. **Acks + retry** — but retries can duplicate, so tag each message with an id and dedupe on the receiver → at-least-once + idempotency.
2. **Persist + re-fetch** — save every message to the DB; on reconnect the client pulls what it missed. The DB is the source of truth, the socket just the fast path. (*This is what the chat app does.*)
3. **Connection State Recovery** — Socket.IO replays missed events after a *brief* drop from a small server-side buffer. Enable explicitly; not durable (a restart wipes it).

INTERVIEW LINE

"Socket.IO guarantees order, not delivery. For durability you persist and re-sync on reconnect, or ack-and-retry with idempotency. I persisted to Mongo, so the socket is the fast path and a reconnect re-fetches anything missed."

12.4 Why persistence over acks?

Acks only confirm the *live* send. They can't help a user who was offline for ten minutes — there's no connection to ack. Persistence covers both: messages live in the DB, so a reconnecting client just re-fetches. Acks are delivery *receipts*, not durability.

13. Acknowledgements (acks)

Request/response layered on top of events.

13.1 What is an ack and its syntax?

A callback the **receiver** runs to confirm receipt; its argument flows back to the sender. The **last function argument** to `emit` is treated as the ack callback.

```
// sender – writes the callback
socket.emit("message:send", data, (res) => {
  console.log("confirmed:", res);
});

// receiver – runs the ack
socket.on("message:send", (data, ack) => {
  save(data);
  ack({ ok: true }); // ← fires the sender's callback
});
```

13.2 Why always add a timeout?

If the receiver never calls `ack` (offline, crashed), the sender waits forever. Bound it:

```
socket.timeout(5000).emit("message:send", data, (err, res) => {
  if (err) retry(); // no ack within 5s
});
// or the promise form:
const res = await socket.timeout(5000).emitWithAck("message:send", data);
```

Note the shape changes to **error-first** `(err, res)` once you add `.timeout()`.

13.3 In a client→server→client flow, where does the ack go?

An ack is **per hop**, not end-to-end. Your cycle is *two separate emits*, so two possible acks. Rule: **whoever emits writes the callback; whoever receives that emit runs `ack()`**, and the reply goes back only *one hop*.

```
sender  —emit→  server    ack1: server → sender    ("server got it")
server  —emit→  recipient  ack2: recipient → server ("recipient got it")
```

There's no single ack from recipient straight back to the sender.

13.4 So how does "delivered ✓✓" work?

You **chain** two single-hop acks. When the recipient's ack fires on the server, the server emits a *new* event back to the original sender:

```
io.to(to).timeout(5000).emit("message:new", saved, (err) => {  
  if (!err) io.to(saved.from).emit("message:delivered", { id: saved._id });  
});
```

The sender listens for `message:delivered` and flips the tick. Socket.IO doesn't do end-to-end for you.

13.5 How does the sender "notice" an ack ran? (the mechanism)

A **correlation id**. When you emit with a callback, Socket.IO assigns the emit a unique id, sends it over the wire, and stores `{ id → callback }`. The receiver's `ack` is a function pre-bound to that id; calling it sends an *ack packet* carrying the same id back. The sender matches the id in its map and runs the stored callback. Same trick as matching an HTTP response to its request.

14. Scaling to many servers

14.1 Why does an in-memory Map of users break at scale?

Production runs several Node processes behind a load balancer. Each has its **own** Map and can't see the others. Alice on server 1, Bob on server 2 → Alice's Map has only Alice, and `io.to(bobId)` on server 1 finds nobody and *silently* does nothing. Classic "works on my machine" bug.

14.2 What fixes it?

- **Redis adapter** (`@socket.io/redis-adapter`) — makes `io.emit` / `io.to()` cross process boundaries via Redis pub/sub. Your emit code doesn't change.
- **Presence in Redis** (with a TTL, so a crashed server leaves no ghosts) instead of a local Map.
- **Sticky sessions** on the load balancer — see below.

14.3 What are sticky sessions and why are they needed?

Once a client lands on server A, it must **keep** hitting server A. The Socket.IO handshake is several HTTP requests carrying one session id; if request 2 hits a different server it rejects the unknown id ("*Session ID unknown*") and the client loops. Config on the LB (nginx `ip_hash` , ALB stickiness cookie), not app code.

COMMON CONFUSION

The Redis adapter does *not* remove the need for sticky sessions. Adapter = broadcasts reaching other servers. Sticky sessions = the handshake's requests reaching the *same* server. You need both.

ESCAPE HATCH

Forcing `transports: ["websocket"]` skips the polling phase, so there's one connection and stickiness is unnecessary — at the cost of the fallback for networks that block WebSocket.

15. When NOT to use WebSockets

15.1 When would you avoid them?

Every open WebSocket is a **held-open, stateful connection** — memory and a file descriptor per client, idle most of the time. That cost drives the answer:

- **Request/response?** Plain HTTP — stateless, cacheable, load-balances trivially.
- **One-way updates?** SSE — cheaper, auto-reconnects, no Upgrade.
- **Server-to-server events?** Webhooks.
- **Serverless / edge?** Often can't hold long connections at all.

SAY THIS

"WebSockets trade simplicity for a persistent, stateful connection per client. If you don't need bidirectional real-time, cheaper tools scale more easily. The connection is the cost."

16. Rapid fire

One-line answers for quick recall.

HTTP status for a successful WS upgrade?	101 Switching Protocols.
Does WS open a second connection?	No — it upgrades the existing TCP connection in place.
ws:// vs wss://	Plaintext vs TLS-encrypted.
Is Socket.IO interoperable with raw ws?	No — different protocol on top of WebSocket.
What transport does Socket.IO fall back to?	HTTP long-polling.
What identifies a TCP connection?	The four-tuple: src IP, src port, dst IP, dst port.
socket.id — connection or person?	Connection. Changes on reconnect; one per tab.
Reach all of one user's tabs?	Room named after their user id; <code>io.to(userId)</code> .
Everyone except the sender?	<code>socket.broadcast.emit</code> .
Where does <code>from</code> come from?	The server's record of the socket, never the client.
Namespace vs room?	Namespace = separate channel/logic; room = dynamic audience.
How are dead sockets detected?	Heartbeat — ping/pong within <code>pingTimeout</code> .
Socket.IO's delivery guarantee?	Order per connection; at-most-once (not delivery).
Make delivery reliable?	Persist + re-fetch, or acks + idempotent retries.
What is an ack?	Receiver-run callback confirming receipt; last fn arg to emit.
Ack across a relay?	Per hop — chain two acks for end-to-end delivered receipts.
Scale WS across servers?	Redis adapter (broadcasts) + sticky sessions (handshake).
SSE vs WebSocket?	SSE one-way over HTTP; WebSocket two-way.
Async socket handler risk?	Unhandled rejection — no error middleware; use try/catch.

Does Redis adapter remove sticky sessions?

No — different problems; you need both.

Built from the chat-app learning sessions · Next.js + Node interview prep · keep alongside the Product Catalog guide.